Percona postgres operator

An honest overview

Yoann La Cancellera

Support engineer

Not a sales guy.
Knows most bugs and all limitations of this operator

Agenda

- Origin and strategy of the project
- Architecture
  - Stack
  - How the images are done and why
- General usage
- Flexibility
  - Extensions
  - Hybrid architecture - avoiding lock-in
- Future plans

Origin and strategy of the project

- Forked from Crunchy operator

- Why ?
  - Stackgres was Java => we're not Java guys
  - Zalando => wasn't deemed the most practical to adapt to our projects
  - Crunchy => **Go** (we've got go people), and Crunchy started its **Crunchy Data Developer program**

Origin and strategy of the project

- Forked from Crunchy operator

- Why ?
  - Stackgres was Java => we're not Java guys
  - Zalando => wasn't deemed the most practical to adapt to our projects
  - Crunchy => Go (we've got go people), and Crunchy started its **Crunchy Data Developer program**

- Our goals back then:
  - You can set any image you want. No restrictions
  - Our images are all based on UBI. No subscriptions
  - Modernizing k8s objects (deployments => statefulsets)

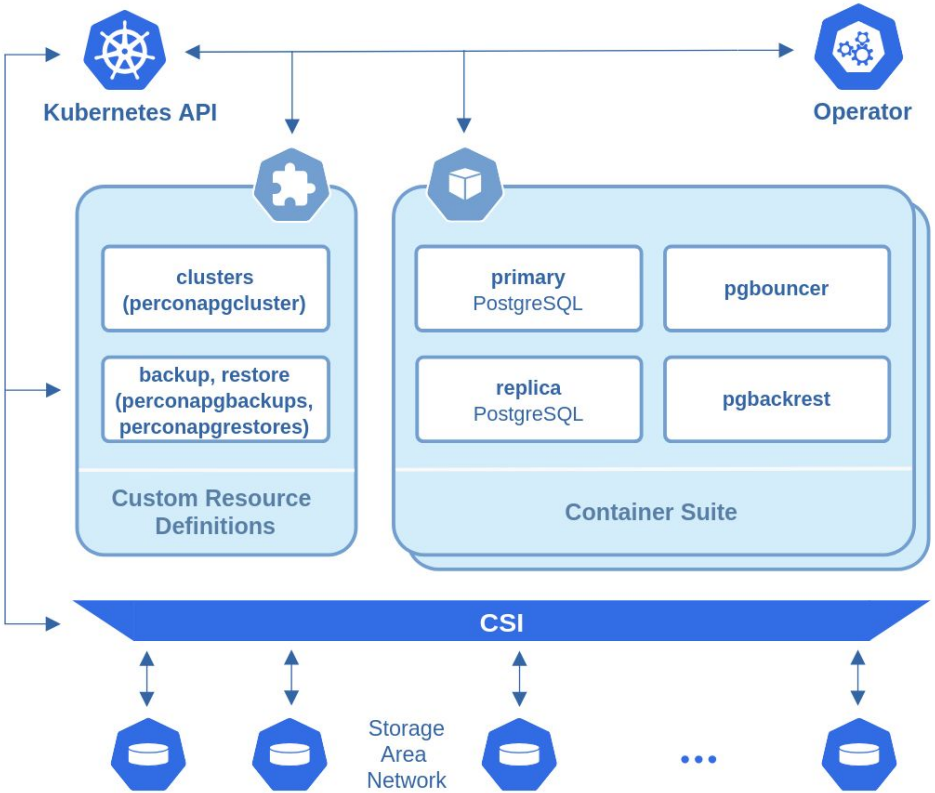
Strategy now

- Aimed to **very large infrastructures** (Back-bone of private-clouds and DBaaS)
- Aimed to **hybrid**: multi-k8s, multi-cloud, k8s-onprem mixes
- Still **fully OSS**
- Just use what works: Patroni, pgbackrest, pgbouncer.

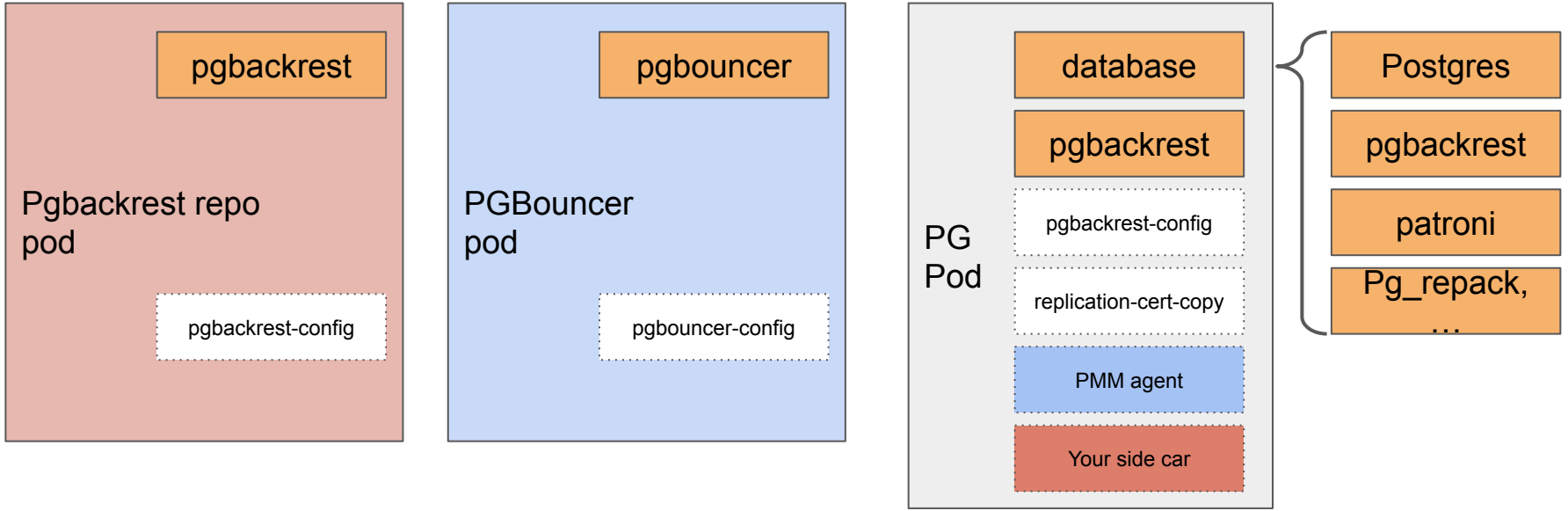
- Small setups / devs: possible, sure. It’s just not our main market.

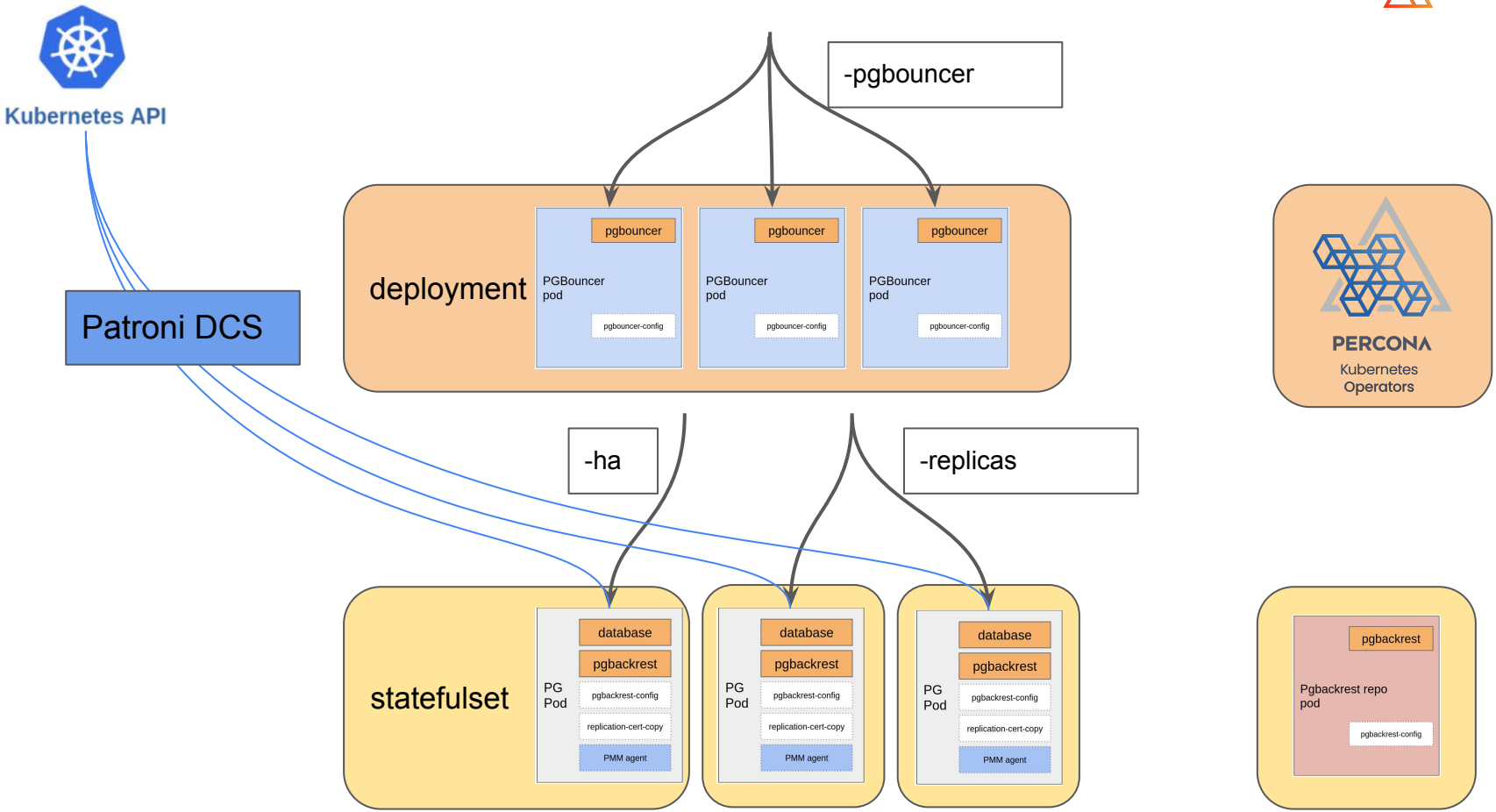
Architecture

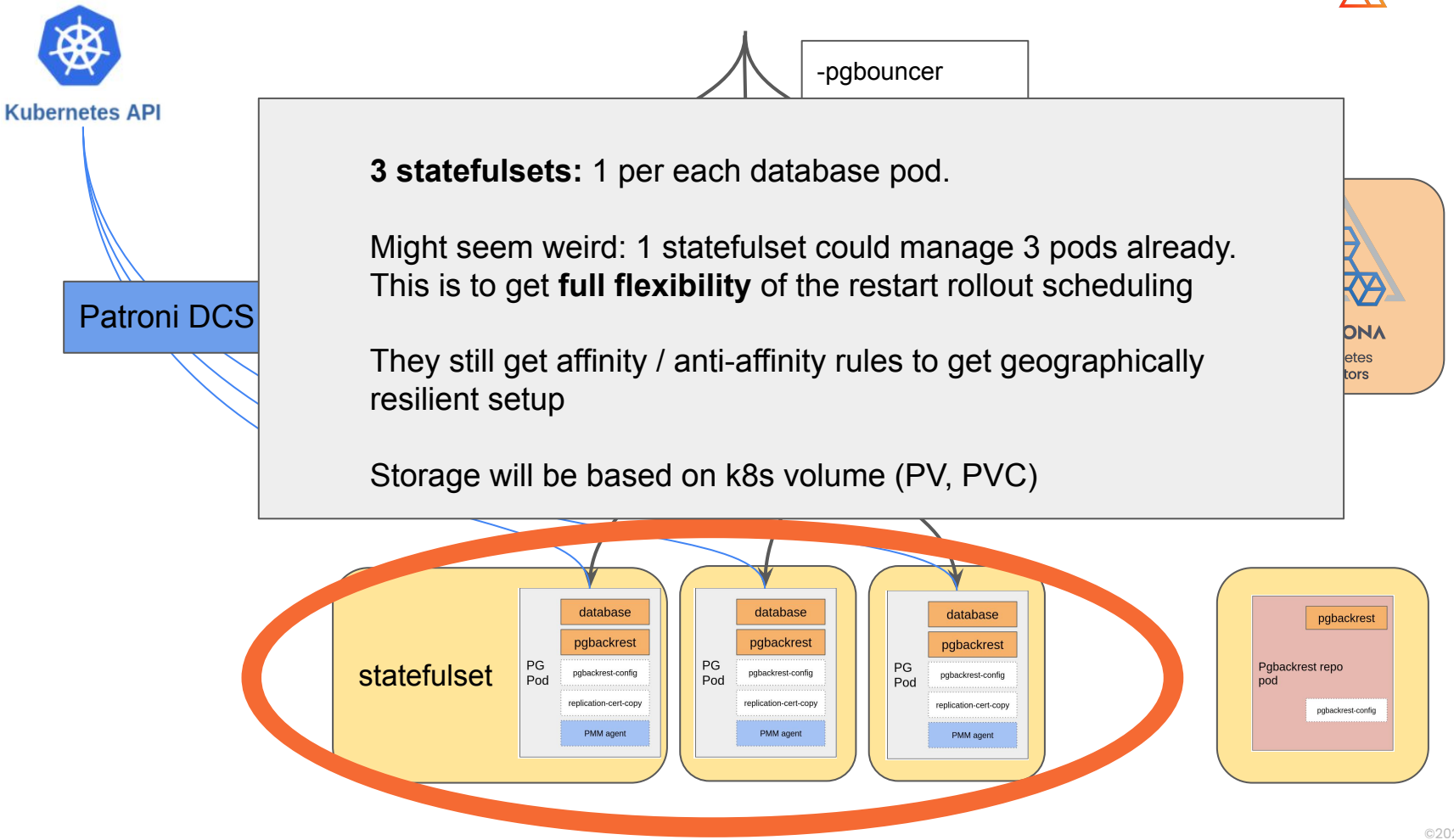
Architecture

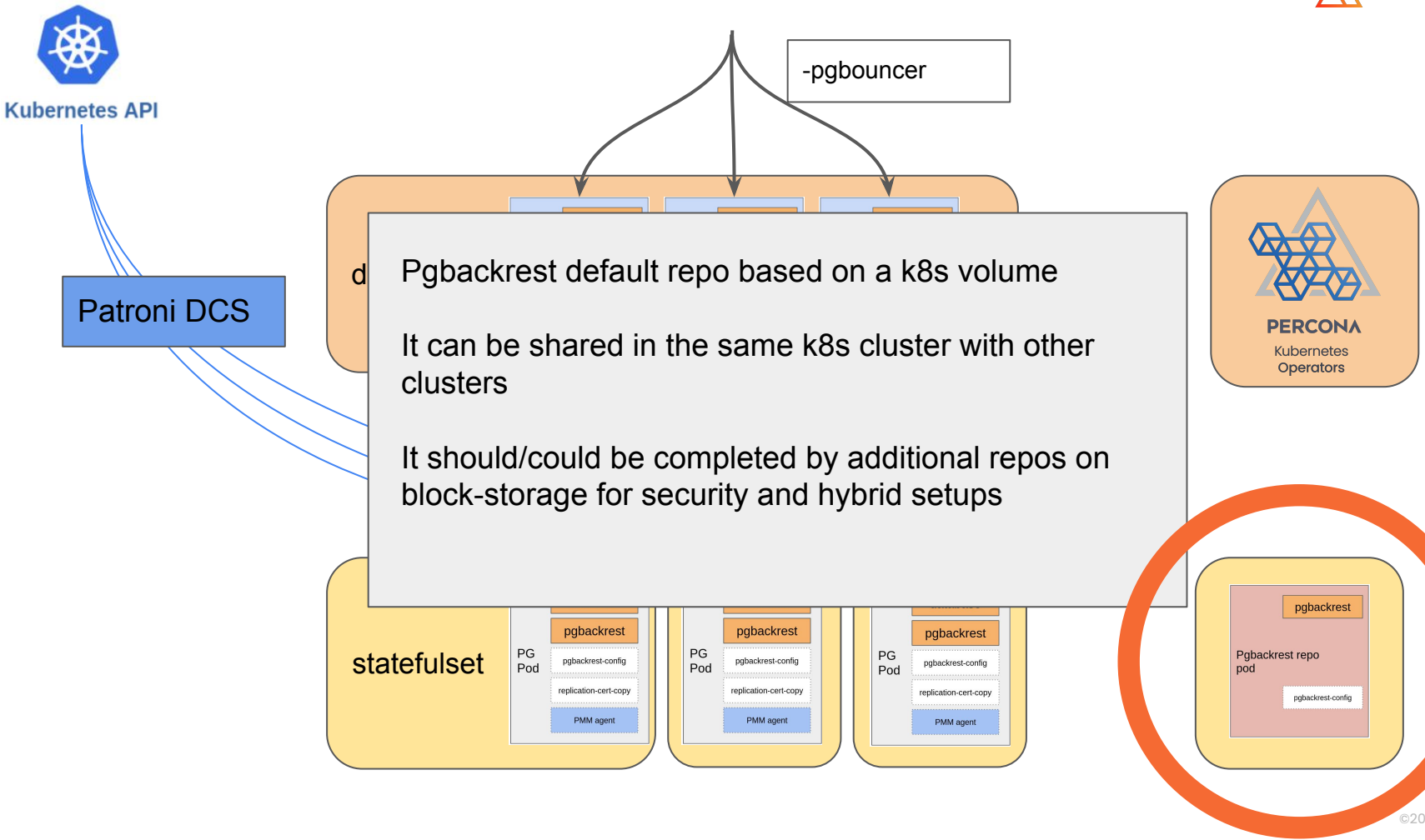


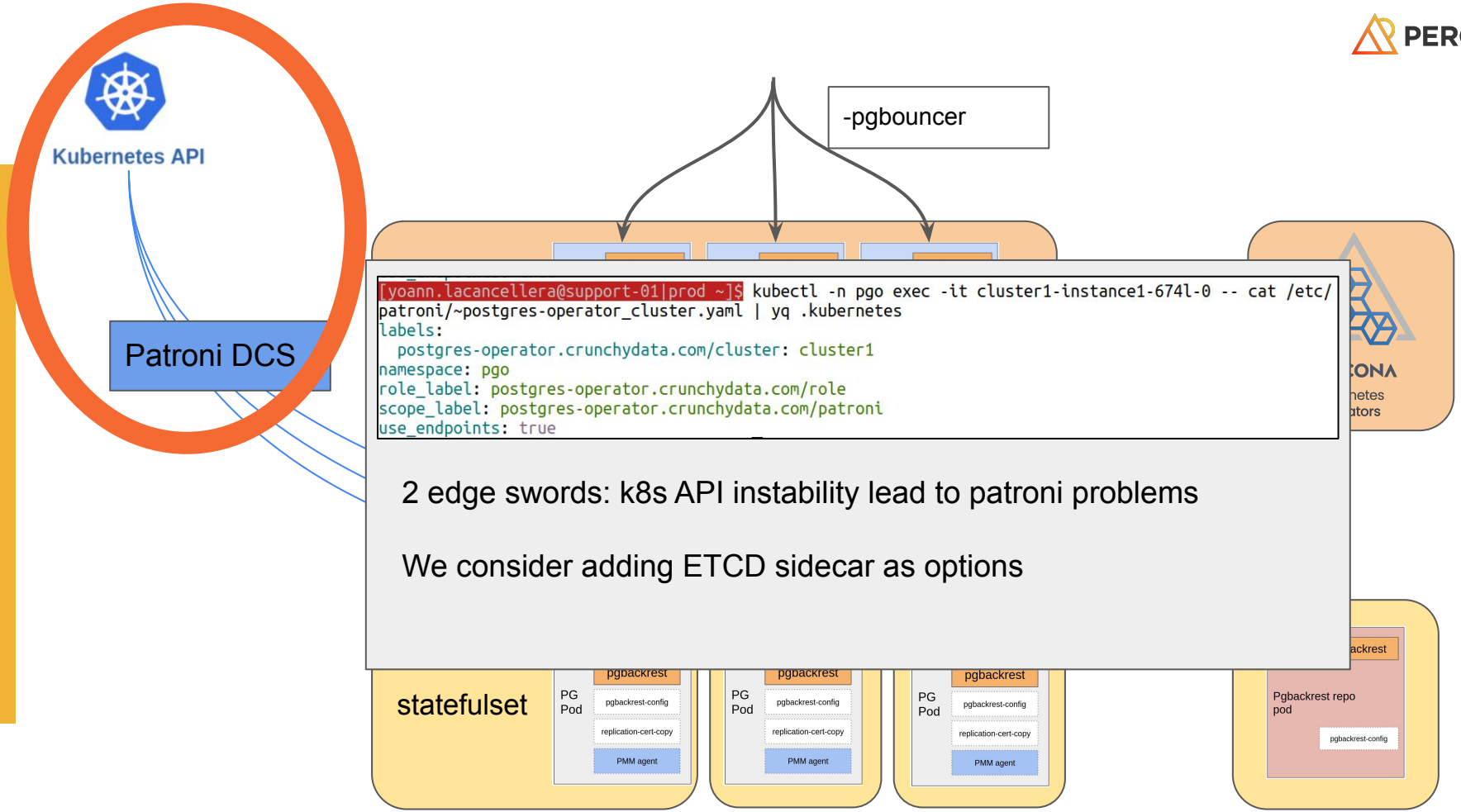
Architecture - pods

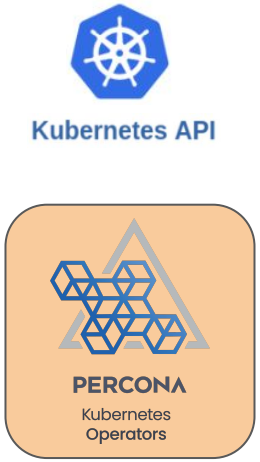


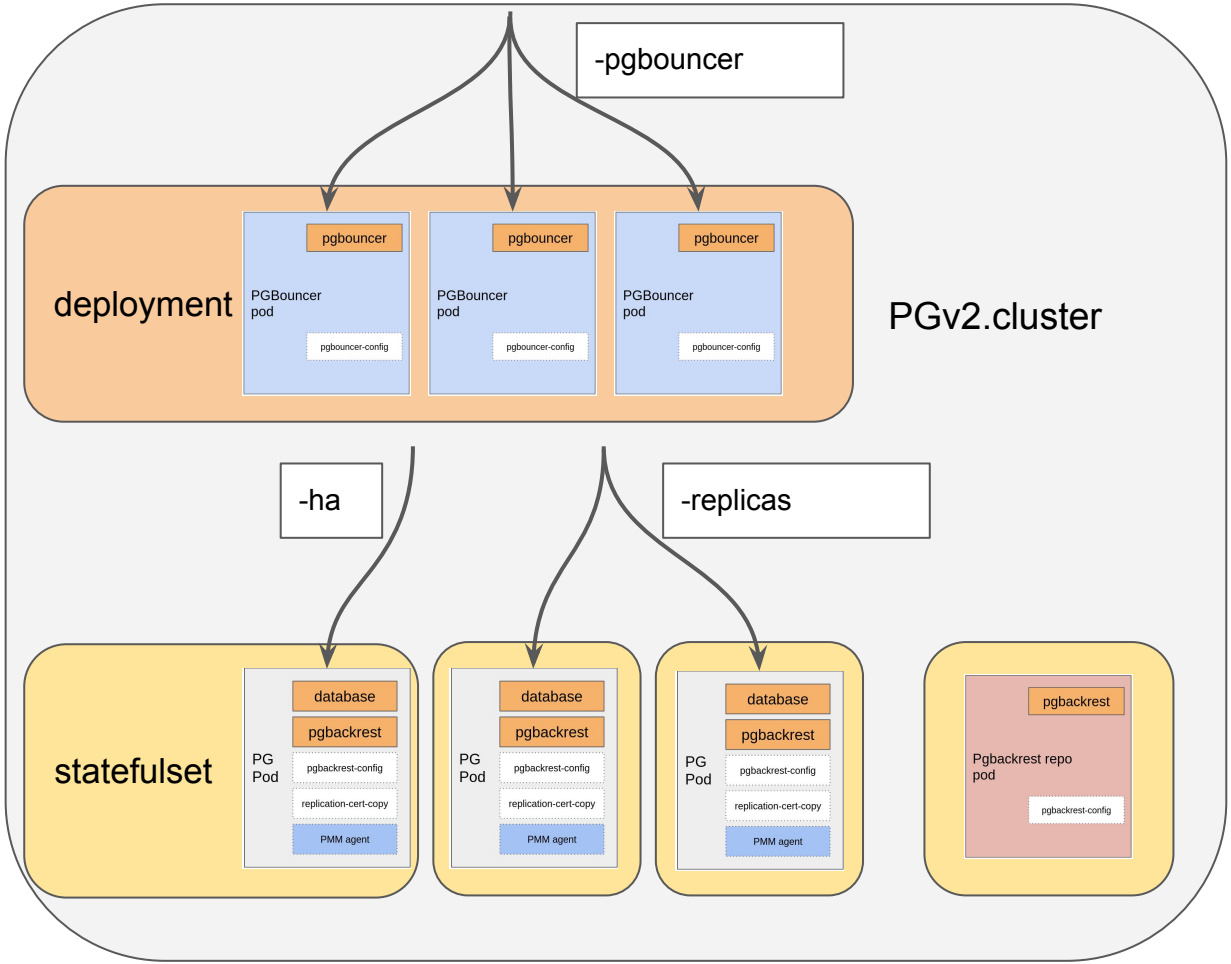


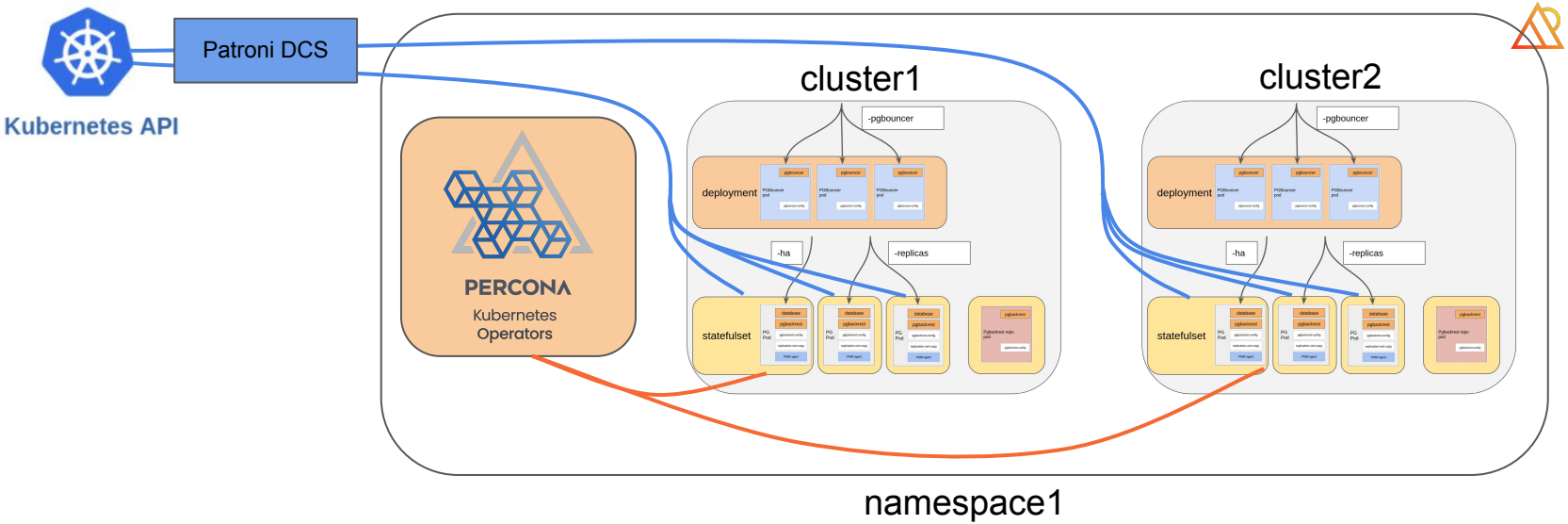












Per-namespace operator:

- Easier to manage resources (cpu,mem), easier to manage parallelism => **stable SLA**
- More difficulty to manage operator versions with their CRD versions (backward compatible though)

Multi-namespace or cluster-wide operator:

- If it tries to handle too many clusters => possible delayed reconciliations
- Upgrades are easier and will benefit every clusters
- Benchmarking in progress

Images

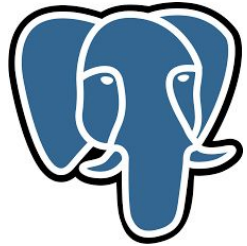
Percona image - operator

- By default, every images are Percona related images.
- All built on **UBI** 8/**9**. Openshift certified. CVE are handled thanks to that.
- **Why**:
  - For end-to-end QA
  - For conformity (e.g: community pgbackrest image runs as root + not built on UBI)
  - To be able to provide custom builds faster
- github.com/percona/percona-docker

```
[yoann.lacancellera@support-01|prod ~]$ kubectl -n pgo get pg cluster1 -o yaml | yq .spec | grep "image:"
    image: docker.io/percona/percona-pgbackrest:2.57.0-1
image: docker.io/percona/percona-distribution-postgresql:17.7-2
  image: docker.io/percona/pmm-client:3.5.0
    image: docker.io/percona/percona-pgbouncer:1.25.0-1
```

Percona image - postgres

- Very tiny rebase adding a few patches **aimed to be merged** to upstream postgres
  - This is how we made pg_tde GA today!
- **You could make your own pg image instead**
- Include pgbouncer, pgbackrest, pgaudit, pg_repack, pg_tde, pg_stat_monitor, ...
- **Drawback**: Glibc related upgrades on k8s



SMGR patches
(Custom storage manager)

pg_tde

Percona distribution
Postgres

General usage

General usage

- Mostly through the **main CR object:**
  - Users, certs
  - Tuning (pgbackrest, patroni, postgres)
  - Architectural choices
  - Observability
  - Resources (CPU, mem, …)

- Many actions can be done manually
  - Pgbackrest related commands
  - Patroni switchover / pause
  - Even debugging (we can disable readyness/liveness probes and fix things)

```
[yoann.lacancellera@support-01|prod ~]$ kubectl -n pgo explain pg.spec.
GROUP:      pgv2.percona.com
KIND:       PerconaPGCluster
VERSION:    v2

FIELD: spec <Object>


DESCRIPTION:
    <empty>
FIELDS:
  autoCreateUserSchema  <boolean>
    Indicates whether schemas are automatically created for the user
    specified in `spec.users` across all databases associated with that user.

  backups       <Object> -required-
    PostgreSQL backup configuration

  crVersion     <string>
    Version of the operator. Update this to new version after operator
    upgrade to apply changes to Kubernetes objects. Default is the latest
    version.

  dataSource    <Object>
    Specifies a data source for bootstrapping the PostgreSQL cluster.

  databaseInitSQL       <Object>
    DatabaseInitSQL defines a ConfigMap containing custom SQL that will
    be run after the cluster is initialized. This ConfigMap must be in the same
    namespace as the cluster.

  expose        <Object>
    Specification of the service that exposes the PostgreSQL primary instance.

  exposeReplicas        <Object>
    Specification of the service that exposes PostgreSQL replica instances

  extensions    <Object>
    The specification of extensions.

  image <string>
    The image name to use for PostgreSQL containers.

  imagePullPolicy       <string>
  enum: Always, Never, IfNotPresent
    ImagePullPolicy is used to determine when Kubernetes will attempt to
    pull (download) container images.
    More info:
    https://kubernetes.io/docs/concepts/containers/images/#image-pull-policy

  imagePullSecrets      <[]Object>
    The image pull secrets used to pull from a private registry
    Changing this value causes all running pods to restart.
    https://k8s.io/docs/tasks/configure-pod-container/pull-image-private-registry/

  initContainer <Object>
    <no description>

  instances     <[]Object> -required-
    Specifies one or more sets of PostgreSQL pods that replicate data for
    this cluster.

  metadata      <Object>
    Metadata contains metadata for custom resources

  openshift     <boolean>
    Whether or not the PostgreSQL cluster is being deployed to an OpenShift
    environment. If the field is unset, the operator will automatically
    detect the environment.

  patroni       <Object>
    <no description>

  pause <boolean>
    Whether or not the PostgreSQL cluster should be stopped.
    When this is true, workloads are scaled to zero and CronJobs
    are suspended.
    Other resources, such as Services and Volumes, remain in place.

  pmm   <Object>
    The specification of PMM sidecars.
```

Flexibility

Extensions

Extensions

- Custom extension are possible
- **Tarball-based setups**

("volumeImage" is based on beta features of k8s)

- Require some packaging to work with your postgres version
- All but Citus and Timescale (because of SMGR incompatibilities)
- Pgaudit, pgvector, pg_repack already here

```
tree ~/pg_cron-1.6.7/
/home/user/pg_cron-1.6.7/
└── usr
    └── pgsql-17
        ├── lib
        │   └── pg_cron.so
        └── share
            └── extension
                ├── pg_cron--1.0--1.1.sql
                ├── pg_cron--1.0.sql
                ├── pg_cron--1.1--1.2.sql
                ├── pg_cron--1.2--1.3.sql
                ├── pg_cron--1.3--1.4.sql
                ├── pg_cron--1.4--1.4-1.sql
                ├── pg_cron--1.4-1--1.5.sql
                ├── pg_cron--1.5--1.6.sql
                └── pg_cron.control
```

Flexibility

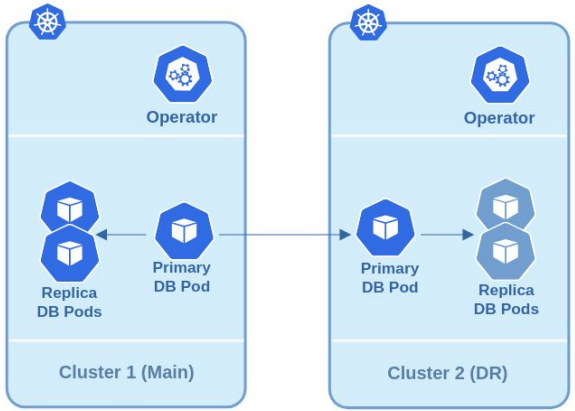
Hybrid setups

Hybrid setups - Patroni

- Based on Patroni standby clusters
  - Will work with **physical streaming replication**
  - “Reinit” will be based on pgbackrest and/or pg_basebackup

- Can work with **on-premise Patroni**: it’s just Patroni.

- All that is required is to be able to connect with _crunchyrepl user and certificates

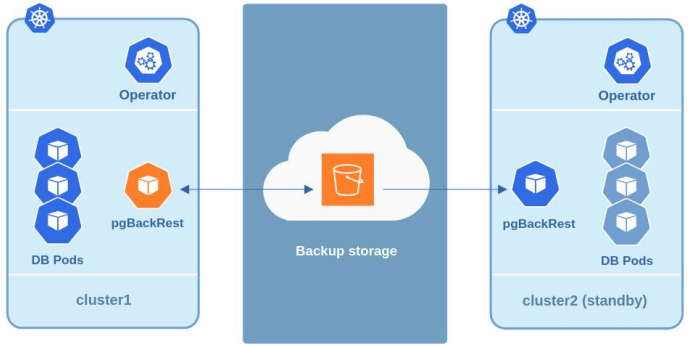



```
spec:
  secrets:
    customTLSSecret:
      name: dr-cluster-cert
    customReplicationTLSSecret:
      name: dr-replication-cert
```

```
  standby:
    enabled: true
    host: main-ha.main-pg.svc
```

Hybrid setups - pgbackrest

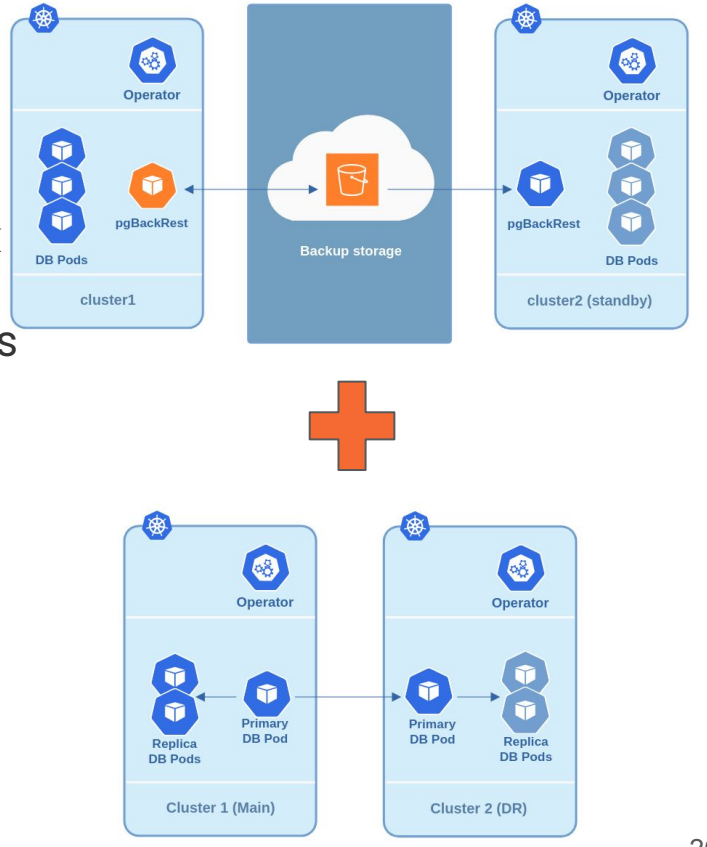
- Based on “pgbackrest archive-get”
- Meant to be used with cloud storage or equivalent
- K8s-volume-based pgbackrest repo: less flexible.
  - The service is not published for external connections (outside k8s) by default
- Can also be used to “**clone**” clusters to a new independent cluster




```
[yoann.lacancellera@support-01|prod ~]$ kubectl -n pgo explain pg.spec.dataSource
GROUP:      pgv2.percona.com
KIND:       PerconaPGCluster
VERSION:    v2

FIELD: dataSource <Object>


DESCRIPTION:
    Specifies a data source for bootstrapping the PostgreSQL cluster.

FIELDS:
  pgbackrest    <Object>
    Defines a pgBackRest cloud-based data source that can be used to
    pre-populate the
    PostgreSQL data directory for a new PostgreSQL cluster using a pgBackRest
    restore.
    The PGBackRest field is incompatible with the PostgresCluster field: only
    one
    data source can be used for pre-populating a new PostgreSQL cluster

  postgresCluster       <Object>
    Defines a pgBackRest data source that can be used to pre-populate the
    PostgreSQL data
    directory for a new PostgreSQL cluster using a pgBackRest restore.
    The PGBackRest field is incompatible with the PostgresCluster field: only
    one
    data source can be used for pre-populating a new PostgreSQL cluster

  volumes       <Object>
    Defines any existing volumes to reuse for this PostgresCluster.
```

Hybrid setups - both!

- **When all is right**: physical replication will work
- **If replication breaks**: pgbackrest takes over as fallback
- Easier switchover between clusters, backups will keep working
- Faster resync of standby clusters



Future plans

Future plans

- Keeping everything open-source ;)
- PG_TDE !
  - Coming in next release **2.9.0**, currently in QA.
  - Next release 2.10.0 should be TDE complete: WAL encryption, backup encryption, ...
- volume snapshots, imageVolumes for extensions too.